Q-CSMA: Queue-Length-Based CSMA/CA Algorithms for Achieving Maximum Throughput and Low Delay in Wireless Networks

Jian Ni, *Member, IEEE*, Bo (Rambo) Tan, *Student Member, IEEE*, and R. Srikant, *Fellow, IEEE*

***Abstract*—Recently, it has been shown that carrier-sense multiple access (CSMA)-type random access algorithms can achieve the maximum possible throughput in ad hoc wireless networks. However, these algorithms assume an idealized continuous-time CSMA protocol where collisions can never occur. In addition, simulation results indicate that the delay performance of these algorithms can be quite bad. On the other hand, although some simple heuristics (such as greedy maximal scheduling) can yield much better delay performance for a large set of arrival rates, in general they may only achieve a fraction of the capacity region. In this paper, we propose a discrete-time version of the CSMA algorithm. Central to our results is a discrete-time distributed randomized algorithm that is based on a generalization of the so-called Glauber dynamics from statistical physics, where multiple links are allowed to update their states in a single timeslot. The algorithm generates collision-free transmission schedules while explicitly taking collisions into account during the control phase of the protocol, thus relaxing the perfect CSMA assumption. More importantly, the algorithm allows us to incorporate heuristics that lead to very good delay performance while retaining the throughput-optimality property.**

***Index Terms*—Carrier-sense multiple access (CSMA), random access, scheduling algorithms, wireless networks.**

I. Introduction

FOR WIRELESS networks with limited resources, efficient resource allocation plays a crucial role in achieving high performance and providing satisfactory quality of service (QoS). In this paper, we study *link scheduling* (or *medium access control*, MAC) for wireless networks, where the links in the network may not be able to transmit simultaneously due to interference. A *scheduling algorithm* (or *MAC protocol*) decides which links can transmit data at each time instant so that no two active links interfere with each other.

Manuscript received November 25, 2009; revised April 05, 2011; accepted August 26, 2011; approved by IEEE/ACM TRANSACTIONS ON NETWORKING Editor A. Proutiere. Date of publication December 08, 2011; date of current version June 12, 2012. This work was supported by the NSF under Grant 07-21286, the ARO MURI under Grants W911NF-07-1-0287 and W911NF-08-1-0233, the AFOSR under Grant FA-9550-08-1-0432, and the DTRA under Grant HDTRA1-08-1-0016. An earlier version of this paper was presented at the IEEE Conference on Computer Communications (INFOCOM), San Diego, CA, March 15–19, 2010.

J. Ni is with the IBM T. J. Watson Research Center, Yorktown Heights, NY 10598 USA (e-mail: nij@us.ibm.com).

B. Tan and R. Srikant are with the Department of Electrical and Computer Engineering, University of Illinois at Urbana–Champaign, Urbana, IL 61801 USA (e-mail: botan2@illinois.edu; rsrikant@illinois.edu).

Color versions of one or more of the figures in this paper are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TNET.2011.2177101

The performance metrics of interest are *throughput* and *delay*. The throughput performance of a scheduling algorithm is characterized by the largest set of arrival rates under which the algorithm can keep the queues in the network stable. The delay performance of a scheduling algorithm can be characterized by the average delay experienced by the packets transmitted in the network. Since many wireless applications nowadays have stringent bandwidth and delay requirements, designing high-performance scheduling algorithms to achieve maximum throughput and low delay is of great importance, which is the focus of this paper. We also want the scheduling algorithms to be distributed and have low complexity/overhead since, in wireless scenarios, normally there is no central entity and the resources at the nodes are very limited.

It is well known that the queue-length-based *Maximum Weighted Scheduling* (MWS) algorithm is *throughput-optimal* [27] in the sense that it can stabilize the network queues for all arrival rates in the capacity region of the network (without explicitly knowing the arrival rates). However, for general interference graphs, MWS requires the network to solve a complex combinatorial optimization problem in each timeslot and, hence, is not implementable in practice.

Maximal scheduling is a low-complexity alternative to MWS, but it may only achieve a small fraction of the capacity region [6], [31]. *Greedy Maximal Scheduling* (GMS), also known as *Longest-Queue-First* (LQF), is another natural low-complexity alternative to MWS that has been observed to achieve good throughput and delay performance. GMS proceeds in a greedy manner by sequentially scheduling a link with the longest queue and disabling all its interfering links. It was shown in [7] that if the network satisfies the so-called *local-pooling* condition, then GMS is throughput-optimal. However, for networks with general topology, GMS may only achieve a fraction of the capacity region [15], [17], [32]. Moreover, while the computational complexity of GMS is low, the signaling and time overhead of decentralization of GMS can increase with the size of the network [17].

Another class of scheduling algorithms are Aloha or *carrier-sense multiple access* (CSMA) type random access algorithms. Under CSMA, a node (transmitter of a link) will sense whether the channel is busy before it transmits a packet. When the node detects the channel is busy, it will wait for a random backoff time. Since CSMA-type algorithms can be easily implemented in a distributed manner, they are widely used in practice (e.g., the IEEE 802.11 MAC protocol). In [4], the authors derived an analytical model to calculate the throughput of a CSMA-type algorithm in multihop wireless networks. They showed that the Markov chain describing the evolution of schedules

1063-6692/$26.00 © 2011 IEEE

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 16,2025 at 12:21:03 UTC from IEEE Xplore. Restrictions apply.

has a product-form stationary distribution under an idealized continuous-time CSMA protocol (which assumes zero propagation/sensing delay and no hidden terminals) where collisions can never occur. Then, the authors proposed a heuristic algorithm to select the CSMA parameters so that the link service rates are equal to the link arrival rates (which were assumed to be known). No proof was given for the convergence of this algorithm. This model was used in [29] to study throughput and fairness issues in wireless ad hoc networks. The insensitivity properties of such a CSMA algorithm have been recently studied in [18].

Based on the results in [4] and [29], a distributed algorithm was developed in [13] to adaptively choose the CSMA parameters to meet the traffic demands without explicitly knowing the arrival rates. The results in [13] make a timescale separation assumption, whereby the CSMA Markov chain converges to its steady-state distribution instantaneously compared to the timescale of adaptation of the CSMA parameters. The authors suggested that this timescale separation assumption can be justified using a stochastic-approximation-type argument, which was later verified in [12] and [20]. Preliminary ideas for a related result were reported in [24], where the authors study distributed algorithms for optical networks, but it is clear that their model also applies to wireless networks with CSMA. In [25], a slightly modified version of the algorithm proposed in [24] was shown to be throughput-optimal. The key idea in [25] is to choose the link weights to be a specific function of the queue lengths to essentially separate the timescales of the link weights and the CSMA dynamics. The results in [25] assume the max-queue-length in the network is known, which is estimated via a message-passing procedure. Similar modifications to 802.11 have been studied in [1] and [30], where the back-pressure on a link is used as the weight. While the results in our paper are most closely related to the works in [4], [13], and [25], we also note important contributions in [5], [8], [21], and [23], which make connections between random access algorithms and stochastic loss networks.

Although the recent results on CSMA-type random access algorithms show throughput optimality, simulation results indicate that the delay performance of these algorithms can be bad and much worse than MWS and GMS. Thus, one of our goals in this paper is to design distributed scheduling algorithms that have low complexity, are provably throughput-optimal, and have good delay performance. Toward this end, we design a discrete-time version of the CSMA-type random access algorithm that achieves the same product-form distribution over schedules as in [4]. Our algorithm generates collision-free data transmission schedules while allowing for collisions during the control phase of the protocol (as in the 802.11 MAC protocol), thus relaxing the perfect CSMA assumption of the algorithms in [4], [13], and [25]. Our approach to modeling collisions is different from the approaches in [14] and [20]. In [20], the authors pointed out that as the transmission probabilities are made small and the transmission lengths are made large, their discrete-time model approximates the continuous-time model with Poisson clocks. In general, though, it is difficult to quantify the throughput difference between these two models. The algorithm in [14] places upper bounds on the CSMA parameters, but the loss in throughput due to this design choice is also hard to quantify. In contrast, we can directly quantify the loss in throughput as the ratio of the duration of the control slot to the duration of the data slot (see Remark 2 in Section IV). More importantly, our formulation allows us to incorporate delay-reduction mechanisms such as GMS in the choice of schedules while retaining the algorithm's throughput-optimality property. It also allows us to handle the hidden and exposed terminal problems associated with wireless networks [2].

We organize the paper as follows. In Section II, we introduce the network model. In Section III, we present the basic scheduling algorithm and show that the (discrete-time) Markov chain of the transmission schedules has a product-form distribution. In Section IV, we present a distributed implementation of the basic scheduling algorithm, called Queue-length-based CSMA/CA (Q-CSMA). In Section V, we propose a hybrid Q-CSMA algorithm that combines Q-CSMA with a distributed procedure that approximates GMS to achieve both maximum throughput and low delay. We evaluate the performance of different scheduling algorithms via simulations in Section VI. The paper is concluded in Section VII.

II. Network Model

We model a (single-channel) wireless network by a graph $G = (V, E)$, where V is the set of nodes and E is the set of links. Nodes are wireless transmitters/receivers. There exists a directed link $(n, m) \in E$ if node m can hear the transmission of node n. We assume that if $(n, m) \in E$, then $(m, n) \in E$.

For any link $i \in E$, we use $\mathcal{C}(i)$ to denote the set of conflicting links (called *conflict set*) of i, i.e., $\mathcal{C}(i)$ is the set of links such that if any one of them is active, then link i cannot be active. The conflict set $\mathcal{C}(i)$ may include the following:

- links that share a common node with link i: This models the *node-exclusive constraint* where two links sharing a common node cannot be active simultaneously;
- links that will cause interference to link i when transmitting: This models the *radio interference constraint* where two links that are close to each other cannot be active simultaneously.

We assume symmetry in the conflict set so that if $i \in \mathcal{C}(j)$, then $j \in \mathcal{C}(i)$. We consider a time-slotted system. A *feasible* (*collision-free*) *schedule* of $G = (V, E)$ is a set of links that can be active at the same time with respect to the conflict set constraint, i.e., no two links in a feasible schedule conflict with each other. Without loss of generality, we assume that all links have unit capacity, i.e., an active link can transmit one packet in one timeslot under a feasible schedule.

A schedule is represented by a vector $\mathbf{x} \in \{0, 1\}^{|E|}$. The ith element of $\mathbf{x}$ is equal to 1 (i.e., $x_i = 1$) if link i is included in the schedule; $x_i = 0$ otherwise. With a little abuse of notation, we also treat $\mathbf{x}$ as a set and write $i \in \mathbf{x}$ if $x_i = 1$. Note that a feasible schedule $\mathbf{x}$ satisfies

$$x_i + x_j \leq 1, \qquad \text{for all } i \in E \text{ and } j \in \mathcal{C}(i). \tag{1}$$

Let $\mathcal{M}$ be the set of all feasible schedules of the network.

A *scheduling algorithm* is a procedure to decide which schedule to be used (which set of links to be activated) in every timeslot for data transmission. In this paper, we focus on the MAC layer so we only consider single-hop traffic. The *capacity region* of the network is the set of all arrival rates $\boldsymbol{\lambda}$ for which there exists a scheduling algorithm that can stabilize the queues, i.e., the queues are bounded in some appropriate

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 16,2025 at 12:21:03 UTC from IEEE Xplore. Restrictions apply.

stochastic sense depending on the arrival model used. For the purposes of this paper, we will assume that if the arrival process is stochastic, then the resulting queue length process admits a Markovian description, in which case stability refers to the positive recurrence of this Markov chain. It is known (e.g., [27]) that the capacity region is given by

$$\Lambda = \{\boldsymbol{\lambda} | \exists \boldsymbol{\mu} \in \mathrm{Co}(\mathcal{M}), \boldsymbol{\lambda} < \boldsymbol{\mu}\}, \tag{2}$$

where $\mathrm{Co}(\mathcal{M})$ is the *convex hull* of the set of feasible schedules in $\mathcal{M}$. When dealing with vectors, inequalities are interpreted componentwise.

We say that a scheduling algorithm is *throughput-optimal*, or achieves the *maximum throughput*, if it can keep the network stable for all arrival rates in Λ.

III. Basic Scheduling Algorithm

We divide each timeslot t into a *control* slot and a *data* slot. (Later, we will further divide the control slot into control mini-slots.) The purpose of the control slot is to generate a collision-free *transmission schedule* $\mathbf{x}(t) \in \mathcal{M}$ used for data transmission in the data slot. To achieve this, the network first selects a set of links that do not conflict with each other, denoted by $\mathbf{m}(t)$. Note that these links also form a feasible schedule, but it is not the schedule used for data transmission. We call $\mathbf{m}(t)$ the *decision schedule* in timeslot t.

Let $\mathcal{M}_0 \subseteq \mathcal{M}$ be the set of possible decision schedules. The network selects a decision schedule according to a randomized procedure, e.g., it selects $\mathbf{m}(t) \in \mathcal{M}_0$ with probability $\alpha(\mathbf{m}(t)) > 0$, where $\sum_{\mathbf{m}(t)\in\mathcal{M}_0} \alpha(\mathbf{m}(t)) = 1$. Then, the transmission schedule is determined as follows. For any link i in $\mathbf{m}(t)$, if no links in $\mathcal{C}(i)$ were active in the previous data slot, then link i is chosen to be *active* with an *activation probability* p_i and *inactive* with probability $1 - p_i$ in the current data slot. If at least one link in $\mathcal{C}(i)$ was active in the previous data slot, then i will be inactive in the current data slot. Any link not selected by $\mathbf{m}(t)$ will maintain its state (active or inactive) from the previous data slot. Conditions on $\mathcal{M}_0$ and the link activation probabilities p_i's will be specified later.

Basic Scheduling Algorithm (in Timeslot t)

1. In the control slot, randomly select a decision schedule $\mathbf{m}(t) \in \mathcal{M}_0$ with probability $\alpha(\mathbf{m}(t))$.
   $\forall i \in \mathbf{m}(t)$:
   If no links in $\mathcal{C}(i)$ were active in the previous data slot, i.e., $\sum_{j\in\mathcal{C}(i)} x_j(t-1) = 0$
   (a) $x_i(t) = 1$ with probability p_i, $0 < p_i < 1$;
   (b) $x_i(t) = 0$ with probability $\bar{p}_i = 1 - p_i$.
   Else
   (c) $x_i(t) = 0$.
   $\forall i \notin \mathbf{m}(t)$:
   (d) $x_i(t) = x_i(t-1)$.
2. In the data slot, use $\mathbf{x}(t)$ as the transmission schedule.

First, we will show that if the transmission schedule used in the previous data slot and the decision schedule selected in the current control slot both are feasible, then the transmission schedule generated in the current data slot is also feasible.

Lemma 1: If $\mathbf{x}(t-1) \in \mathcal{M}$ and $\mathbf{m}(t) \in \mathcal{M}$, then $\mathbf{x}(t) \in \mathcal{M}$.

Proof: Note that $\mathbf{x} \in \mathcal{M}$ if and only if $\forall i \in E$ such that $x_i = 1$, we have $x_j = 0$ for all $j \in \mathcal{C}(i)$.

Now consider any $i \in E$ such that $x_i(t) = 1$. If $i \notin \mathbf{m}(t)$, then we know $x_i(t-1) = x_i(t) = 1$, and since $\mathbf{x}(t-1) \in \mathcal{M}$, we have $\forall j \in \mathcal{C}(i)$, $x_j(t-1) = 0$. In addition, if $j \notin \mathbf{m}(t)$, then $x_j(t) = x_j(t-1) = 0$ based on Step (d) of the scheduling algorithm. Otherwise if $j \in \mathbf{m}(t)$, then since $i \in \mathcal{C}(j)$ and $x_i(t-1) = 1$, $x_j(t) = 0$ based on Step (c).

If $i \in \mathbf{m}(t)$, from the scheduling algorithm we have $x_i(t) = 1$ only if $x_j(t-1) = 0, \forall j \in \mathcal{C}(i)$. Since $i \in \mathbf{m}(t)$ and $\mathbf{m}(t)$ is feasible, we know $\mathcal{C}(i) \cap \mathbf{m}(t) = \emptyset$. Therefore, for any $j \in \mathcal{C}(i)$, $x_j(t) = x_j(t-1) = 0$ based on Step (d). ∎

Since $\mathbf{x}(t)$ only depends on the previous state $\mathbf{x}(t-1)$ and a randomly selected decision schedule $\mathbf{m}(t)$, $\mathbf{x}(t)$ evolves as a discrete-time Markov chain (DTMC). Next, we will derive the transition probabilities between the states.

Lemma 2: A state $\mathbf{x} \in \mathcal{M}$ can make a transition to a state $\mathbf{x}' \in \mathcal{M}$ if and only if $\mathbf{x} \cup \mathbf{x}' \in \mathcal{M}$ and there exists a decision schedule $\mathbf{m} \in \mathcal{M}_0$ such that

$$\mathbf{x} \triangle \mathbf{x}' = (\mathbf{x} \setminus \mathbf{x}') \cup (\mathbf{x}' \setminus \mathbf{x}) \subseteq \mathbf{m}$$

and in this case the transition probability from $\mathbf{x}$ to $\mathbf{x}'$ is

$$P(\mathbf{x}, \mathbf{x}') = \sum_{\mathbf{m}\in\mathcal{M}_0 : \mathbf{x}\triangle\mathbf{x}' \subseteq \mathbf{m}} \alpha(\mathbf{m}) \left(\prod_{l\in\mathbf{x}\setminus\mathbf{x}'} \bar{p}_l\right) \left(\prod_{k\in\mathbf{x}'\setminus\mathbf{x}} p_k\right) \times \left(\prod_{i\in\mathbf{m}\cap(\mathbf{x}\cap\mathbf{x}')} p_i\right) \left(\prod_{j\in\mathbf{m}\setminus(\mathbf{x}\cup\mathbf{x}')\setminus\mathcal{C}(\mathbf{x}\cup\mathbf{x}')} \bar{p}_j\right). \tag{3}$$

Proof: (*Necessity*) Suppose $\mathbf{x}$ is the current state and $\mathbf{x}'$ is the next state. $\mathbf{x} \setminus \mathbf{x}' = \{l : \mathbf{x}_l = 1, \mathbf{x}'_l = 0\}$ is the set of links that change their state from 1 (active) to 0 (inactive). $\mathbf{x}' \setminus \mathbf{x} = \{k : \mathbf{x}_k = 0, \mathbf{x}'_k = 1\}$ is the set of links that change their state from 0 to 1. From the scheduling algorithm, a link can change its state only if it belongs to the decision schedule. Therefore, $\mathbf{x}$ can make a transition to $\mathbf{x}'$ only if there exists an $\mathbf{m} \in \mathcal{M}_0$ such that the symmetric difference $\mathbf{x} \triangle \mathbf{x}' = (\mathbf{x} \setminus \mathbf{x}') \cup (\mathbf{x}' \setminus \mathbf{x}) \subseteq \mathbf{m}$. In addition, since $(\mathbf{x}\cap\mathbf{x}')\cup(\mathbf{x}\setminus\mathbf{x}') = \mathbf{x} \in \mathcal{M}$, $(\mathbf{x}\cap\mathbf{x}')\cup(\mathbf{x}'\setminus\mathbf{x}) = \mathbf{x}' \in \mathcal{M}$, and $(\mathbf{x} \setminus \mathbf{x}') \cup (\mathbf{x}' \setminus \mathbf{x}) = \mathbf{x} \triangle \mathbf{x}' \in \mathcal{M}$, we have $\mathbf{x} \cup \mathbf{x}' = (\mathbf{x} \setminus \mathbf{x}') \cup (\mathbf{x}' \setminus \mathbf{x}) \cup (\mathbf{x} \cap \mathbf{x}') \in \mathcal{M}$.

(*Sufficiency*) Now suppose $\mathbf{x} \cup \mathbf{x}' \in \mathcal{M}$ and there exists an $\mathbf{m} \in \mathcal{M}_0$ such that $\mathbf{x} \triangle \mathbf{x}' \subseteq \mathbf{m}$. Given $\mathbf{m}$ is the selected decision schedule, we can calculate the (conditional) probability that $\mathbf{x}$ makes a transition to $\mathbf{x}'$ by dividing the links in $\mathbf{m}$ into the following five cases.

1) $l \in \mathbf{x} \setminus \mathbf{x}'$: Link l decides to change its state from 1 to 0. This occurs with probability $\bar{p}_l$ based on Step (b) in the scheduling algorithm.
2) $k \in \mathbf{x}' \setminus \mathbf{x}$: Link k decides to change its state from 0 to 1. This occurs with probability p_k based on Step (a).
3) $i \in \mathbf{m} \cap (\mathbf{x} \cap \mathbf{x}')$: Link i decides to keep its state 1. This occurs with probability p_i based on Step (a).
4) $e \in \mathbf{m} \cap \mathcal{C}(\mathbf{x})$ where $\mathcal{C}(\mathbf{x}) = \cup_{l\in\mathbf{x}}\mathcal{C}(l)$: Link e has to keep its state 0. This occurs with probability 1 based on Step (c).
5) $j \in \mathbf{m} \setminus (\mathbf{x} \cup \mathbf{x}') \setminus \mathcal{C}(\mathbf{x})$: Link j decides to keep its state 0. This occurs with probability $\bar{p}_j$ based on Step (b).

Note that $\mathbf{m} \cap \mathcal{C}(\mathbf{x}' \setminus \mathbf{x}) = \emptyset$ because $\mathbf{x}' \setminus \mathbf{x} \subseteq \mathbf{m}$, we have $\mathbf{m} \setminus (\mathbf{x} \cup \mathbf{x}') \setminus \mathcal{C}(\mathbf{x}) = \mathbf{m} \setminus (\mathbf{x} \cup \mathbf{x}') \setminus \mathcal{C}(\mathbf{x} \cup \mathbf{x}')$. Since each

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 16,2025 at 12:21:03 UTC from IEEE Xplore. Restrictions apply.

link in $\mathbf{m}$ makes its decision independently of each other, we can multiply these probabilities together. Summing over all possible decision schedules, we get the total transition probability from $\mathbf{x}$ to $\mathbf{x}'$ given in (3). ■

Proposition 1: A necessary and sufficient condition for the DTMC of the transmission schedules to be irreducible and aperiodic is

$$\cup_{\mathbf{m}\in\mathcal{M}_0}\mathbf{m} = E \tag{4}$$

and in this case the DTMC is reversible and has the following product-form stationary distribution:

$$\pi(\mathbf{x}) = \frac{1}{Z}\prod_{i\in\mathbf{x}}\frac{p_i}{\bar{p}_i} \tag{5}$$

$$Z = \sum_{\mathbf{x}\in\mathcal{M}}\prod_{i\in\mathbf{x}}\frac{p_i}{\bar{p}_i}. \tag{6}$$

Proof: If $\cup_{\mathbf{m}\in\mathcal{M}_0}\mathbf{m} \neq E$, suppose $l \notin \cup_{\mathbf{m}\in\mathcal{M}_0}\mathbf{m}$, then from state $\mathbf{0}$ the DTMC will never reach a feasible schedule including l. On the other hand, if $\cup_{\mathbf{m}\in\mathcal{M}_0}\mathbf{m} = E$, then using Lemma 2, it is easy to verify that state $\mathbf{0}$ can reach any other state $\mathbf{x} \in \mathcal{M}$ with positive probability in a finite number of steps, and vice versa. To see this, suppose $\mathbf{x} = \{l_1, l_2, \ldots, l_m\}$. Define $\mathbf{x}_j = \{l_1, \ldots, l_j\}$ for $j = 0, \ldots, m$. Note that $\mathbf{x}_0 = \mathbf{0}$ and $\mathbf{x}_m = \mathbf{x}$. Now for $0 \leq j \leq m-1$, $\mathbf{x}_j \cup \mathbf{x}_{j+1} = \mathbf{x}_{j+1} \in \mathcal{M}$ and $\mathbf{x}_j \triangle \mathbf{x}_{j+1} = \{l_{j+1}\}$. Since $\cup_{\mathbf{m}\in\mathcal{M}_0}\mathbf{m} = E$, there exists an $\mathbf{m} \in \mathcal{M}_0$ such that $\{l_{j+1}\} \subseteq \mathbf{m}$. Then, by Lemma 2, $\mathbf{x}_j$ can make a transition to $\mathbf{x}_{j+1}$ with positive probability as given in (3), hence $\mathbf{0}$ can reach $\mathbf{x}$ with positive probability in a finite number of steps. The reverse argument is similar. Therefore, the DTMC is irreducible and aperiodic.

If $\mathbf{x}$ can make a transition to $\mathbf{x}'$, then we can check that the distribution in (5) satisfies the detailed balance equation

$$\begin{aligned}\pi(\mathbf{x})P(\mathbf{x},\mathbf{x}') &= \frac{1}{Z}\sum_{\mathbf{m}\in\mathcal{M}_0:\mathbf{x}\triangle\mathbf{x}'\subseteq\mathbf{m}}\alpha(\mathbf{m})\left(\frac{\prod_{k\in\mathbf{x}\cup\mathbf{x}'}p_k}{\prod_{l\in\mathbf{x}\cap\mathbf{x}'}\bar{p}_l}\right)\\ &\quad\times\left(\prod_{i\in\mathbf{m}\cap(\mathbf{x}\cap\mathbf{x}')}p_i\right)\left(\prod_{j\in\mathbf{m}\setminus(\mathbf{x}\cup\mathbf{x}')\setminus\mathcal{C}(\mathbf{x}\cup\mathbf{x}')}\bar{p}_j\right)\\ &= \pi(\mathbf{x}')P(\mathbf{x}',\mathbf{x})\end{aligned} \tag{7}$$

hence the DTMC is reversible and (5) is indeed its stationary distribution (see, for example, [16]). ■

A. Comments on Throughput Optimality

Based on the product-form distribution in Proposition 1, one can then proceed as in [13] (under a timescale separation assumption) or as in [11] and [25] (without such an assumption, but with certain choices of the weight function) to establish throughput optimality of the scheduling algorithm. Instead of pursuing such a proof here, we point out the intuition behind the proof of throughput optimality under the timescale separation assumption in [13].

We associate each link $i \in E$ with a nonnegative weight $w_i(t)$ in timeslot t. Recall that MWS selects a maximum-weight schedule $\mathbf{x}^*(t)$ in every timeslot t such that

$$\sum_{i\in\mathbf{x}^*(t)} w_i(t) = \max_{\mathbf{x}\in\mathcal{M}}\sum_{i\in\mathbf{x}} w_i(t). \tag{8}$$

Let $q_i(t)$ be the queue length of link i at the beginning of timeslot t. It was proved in [27] that MWS is throughput-optimal if we let $w_i(t) = q_i(t)$. This result was generalized in [9] as follows. For all $i \in E$, let link weight $w_i(t) = f_i(q_i(t))$, where $f_i : [0,\infty] \to [0,\infty]$ are functions that satisfy the following conditions.

1) $f_i(q_i)$ is a nondecreasing, continuous function with $\lim_{q_i\to\infty} f_i(q_i) = \infty$.
2) Given any $M_1 > 0$, $M_2 > 0$ and $0 < \epsilon < 1$, there exists a $Q < \infty$, such that for all $q_i > Q$ and $\forall i$, we have

$$(1-\epsilon)f_i(q_i) \leq f_i(q_i - M_1) \leq f_i(q_i + M_2) \leq (1+\epsilon)f_i(q_i).$$

The following result was established in [9].

Theorem 1: For a scheduling algorithm, if given any ϵ and δ, $0 < \epsilon, \delta < 1$, there exists a $B > 0$ such that in any timeslot t, with probability greater than $1-\delta$, the scheduling algorithm chooses a schedule $\mathbf{x}(t) \in \mathcal{M}$ that satisfies

$$\sum_{i\in\mathbf{x}(t)} w_i(t) \geq (1-\epsilon)\max_{\mathbf{x}\in\mathcal{M}}\sum_{i\in\mathbf{x}} w_i(t) \tag{9}$$

whenever $\|\mathbf{q}(t)\| > B$, where $\mathbf{q}(t) = (q_i(t) : i \in E)$. Then, the scheduling algorithm is throughput-optimal.

If we choose the link activation probability

$$p_i = \frac{e^{w_i(t)}}{e^{w_i(t)}+1} \qquad \forall i \in E \tag{10}$$

then (5) becomes

$$\pi(\mathbf{x}) = \frac{1}{Z}\prod_{i\in\mathbf{x}}\frac{p_i}{\bar{p}_i} = \frac{1}{Z}\prod_{i\in\mathbf{x}} e^{w_i(t)} = \frac{e^{\sum_{i\in\mathbf{x}} w_i(t)}}{Z}. \tag{11}$$

Hence, the (steady-state) probability of choosing a schedule is proportional to the exponent of its weight, so schedules with larger weight will be selected with higher probability. This is the intuition behind our proof.

By choosing the link weight functions f_i's appropriately, we can make the DTMC of the transmission schedules converge much faster compared to the dynamics of the link weights. For example, $f_i(q_i) = \alpha q_i$ with a small α is suggested as a heuristic to satisfy the timescale separation assumption in [13], and $f_i(q_i) = \log\log(q_i + e)$ is used in the proof of throughput optimality in [25] to essentially separate the timescales. Here, as in [13], we simply assume that the DTMC is in the steady state in every timeslot.

Proposition 2: Suppose the basic scheduling algorithm satisfies $\cup_{\mathbf{m}\in\mathcal{M}_0}\mathbf{m} = E$ and hence has the product-form stationary

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 16,2025 at 12:21:03 UTC from IEEE Xplore. Restrictions apply.

distribution. Let $p_i = e^{w_i(t)}/(e^{w_i(t)}+1)$, $\forall i \in E$. Then, the scheduling algorithm is throughput-optimal.

Proof: We prove the proposition using Theorem 1. Now given any ϵ and δ such that $0 < \epsilon, \delta < 1$. Let $w^*(t) := \max_{\mathbf{x}\in\mathcal{M}} \sum_{i\in\mathbf{x}} w_i(t)$. Define

$$\mathcal{X} := \left\{ \mathbf{x} \in \mathcal{M} : \sum_{i\in\mathbf{x}} w_i(t) < (1-\epsilon) w^*(t) \right\}.$$

Since the DTMC has the product-form stationary distribution in (11), we have

$$\begin{aligned} \pi(\mathcal{X}) &= \sum_{\mathbf{x}\in\mathcal{X}} \pi(\mathbf{x}) = \sum_{\mathbf{x}\in\mathcal{X}} \frac{e^{\sum_{i\in\mathbf{x}} w_i(t)}}{Z} \\ &\le \frac{|\mathcal{X}| e^{(1-\epsilon)w^*(t)}}{Z} < \frac{2^{|E|}}{e^{\epsilon w^*(t)}} \end{aligned} \tag{12}$$

where (12) is true because $|\mathcal{X}| \le |\mathcal{M}| \le 2^{|E|}$, and $Z > e^{\max_{\mathbf{x}\in\mathcal{M}} \sum_{i\in\mathbf{x}} w_i(t)} = e^{w^*(t)}$. Therefore, if

$$w^*(t) > \frac{1}{\epsilon}\left(|E| \log 2 + \log \frac{1}{\delta} \right) \tag{13}$$

then $\pi(\mathcal{X}) < \delta$. Since $w^*(t)$ is a continuous, nondecreasing function of $q_i(t)$'s, with $\lim_{\|\mathbf{q}(t)\|\to\infty} w^*(t) = \infty$, there exits a $B > 0$ such that whenever $\|\mathbf{q}(t)\| > B$, (13) holds so $\pi(\mathcal{X}) < \delta$. Hence, the scheduling algorithm satisfies the condition of Theorem 1 and is throughput-optimal. ∎

Remark 1: Our algorithm is a generalization of Glauber dynamics where multiple links are allowed to make decisions in a single timeslot [28].

IV. Distributed Implementation: Q-CSMA

In this section, we present a distributed implementation of the basic scheduling algorithm. The key idea is to develop a distributed randomized procedure to select a (feasible) decision schedule in the control slot. To achieve this, we further divide the control slot into control mini-slots. Note that once a link knows that it is included in the decision schedule, it can determine its state in the data slot based on its carrier-sensing information (i.e., whether its conflicting links were active in the previous data slot) and activation probability. We call this implementation *Q-CSMA* since the activation probability of a link is determined by its queue length (to achieve maximum throughput, see Section III-A), and collisions of data packets are avoided via carrier sensing and the exchange of control messages.

At the beginning of each timeslot, every link i will select a random backoff time. Link i will send a message announcing its INTENT to make a decision at the expiry of this backoff time subject to the constraints described in the following.

Q-CSMA Algorithm (at Link i in Timeslot t)

1. Link i selects a random (integer) backoff time T_i uniformly in $[0, W-1]$ and waits for T_i control mini-slots.
2. IF link i hears an INTENT message from a link in $\mathcal{C}(i)$ before the (T_i+1)th control mini-slot, i will not be included in $\mathbf{m}(t)$ and will not transmit an INTENT message. Link i will set $x_i(t) = x_i(t-1)$.
3. IF link i does not hear an INTENT message from any link in $\mathcal{C}(i)$ before the (T_i+1)th control mini-slot, it will send (broadcast) an INTENT message to all links in $\mathcal{C}(i)$ at the beginning of the (T_i+1)th control mini-slot.
   - If there is a collision (i.e., if there is another link in $\mathcal{C}(i)$ transmitting an INTENT message in the same mini-slot), link i will not be included in $\mathbf{m}(t)$ and will set $x_i(t) = x_i(t-1)$.
   - If there is no collision, link i will be included in $\mathbf{m}(t)$ and decide its state as follows:
     if no links in $\mathcal{C}(i)$ were active in the previous data slot
       $x_i(t) = 1$ with probability p_i, $\quad 0 < p_i < 1$
       $x_i(t) = 0$ with probability $\bar{p}_i = 1 - p_i$.
     else
       $x_i(t) = 0$.
4. If $x_i(t) = 1$, link i will transmit a packet in the data slot.

Lemma 3: $\mathbf{m}(t)$ produced by Q-CSMA is a feasible schedule. Let $\mathcal{M}_0$ be the set of all decision schedules produced by Q-CSMA. If the window size $W \ge 2$, then $\cup_{\mathbf{m}\in\mathcal{M}_0} \mathbf{m} = E$.

Proof: Under Q-CSMA, link i will be included in the decision schedule $\mathbf{m}(t)$ if and only if it successfully sends an INTENT message to all links in $\mathcal{C}(i)$ without a collision in the control slot. This will "silence" the links in $\mathcal{C}(i)$ so those links will not be included in $\mathbf{m}(t)$. Hence, $\mathbf{m}(t)$ is feasible.

Now for any maximal schedule $\mathbf{m}$ (a schedule is *maximal* if no additional links can be added to the schedule without violating its feasibility), note that $\mathbf{m}$ will be selected in the control slot if $T_i = 0, \forall i \in \mathbf{m}$, and $T_j = 1, \forall j \notin \mathbf{m}$. This occurs with positive probability if $W \ge 2$ because

$$\alpha(\mathbf{m}) \ge \Pr\{T_i = 0, \forall i \in \mathbf{m};\ T_j = 1,\ \forall j \notin \mathbf{m}\} = \prod_{i\in E} \frac{1}{W} > 0.$$

Since the set of all maximal schedules will include all links, $\cup_{\mathbf{m}\in\mathcal{M}_0} \mathbf{m} = E$ if $W \ge 2$. ∎

Combining Lemma 3 and Propositions 1 and 2, we have the following result.

Proposition 3: Q-CSMA has the product-form distribution given in Proposition 1 if $W \ge 2$. Furthermore, it is throughput-optimal if we let $p_i = e^{w_i(t)}/(e^{w_i(t)}+1)$, $\forall i \in E$.

Remark 2: A control slot of Q-CSMA consists of W mini-slots, and each link needs to send at most one INTENT message. Hence, Q-CSMA has constant (and low) signaling/time overhead, independent of the size of the network. Suppose the duration of a data slot is D mini-slots. Taking control overhead into account, Q-CSMA can achieve $D/(D+W)$ of the capacity region, which approaches the full capacity when $W \ll D$.

Remark 3: We can slightly modify Q-CSMA as follows. In Step 3, if link i does not hear an INTENT message from any link in $\mathcal{C}(i)$ before the (T_i+1)th control mini-slot, i will send an INTENT message to all links in $\mathcal{C}(i)$ at the beginning of the (T_i+1)th control mini-slot with some (positive) probability

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 16,2025 at 12:21:03 UTC from IEEE Xplore. Restrictions apply.

$\beta_i < 1$. In this case, Q-CSMA achieves the product-form distribution even for $W = 1$.[1]

When describing the Q-CSMA algorithm, we treat every link as an entity, while in reality each link consists of a sender node and a receiver node. Both carrier sensing and transmission of data/control packets are actually conducted by the nodes. In Appendix-A, we provide details to implement Q-CSMA based on the nodes. Such an implementation also allows us to handle the hidden and exposed terminal problems associated with wireless networks [2]; see Appendix-B.

V. Low-Delay Hybrid Q-CSMA Algorithm

By Little's law, the long-term average queueing delay experienced by the packets is proportional to the long-term average queue length in the network. In our simulations (Section VI), we find that the delay performance of Q-CSMA can be quite bad when the traffic intensity is high (this is also true in simulations of continuous-time CSMA algorithms) and much worse than GMS. However, GMS is a centralized algorithm and is not throughput-optimal in general. We are therefore motivated to design a distributed scheduling algorithm that can combine the advantages of both Q-CSMA (for achieving maximum throughput) and GMS (for achieving low delay). We first develop a distributed algorithm to approximate GMS, which we call D-GMS.

The basic idea of D-GMS is to assign smaller backoff times to links with larger queue lengths. However, to handle cases where two or more links in a neighborhood have the same queue length, some collision resolution mechanism is incorporated in D-GMS. Furthermore, we have conducted extensive simulations to understand how to reduce the control overhead required to implement D-GMS while maintaining the ability to control the network when the queue lengths become large. Based on these simulations, we conclude that it is better to use the log of the queue lengths (rather than the queue lengths themselves) to determine the channel access priority of the links. The resulting D-GMS algorithm is described as follows.

D-GMS Algorithm (at Link i in Timeslot t)

1. Link i selects a random backoff time

$$T_i = W \times \lfloor B - \log_b (q_i(t)+1) \rfloor^+ + \text{Uniform}[0, W-1]$$

   and waits for T_i control mini-slots.
2. IF link i hears an RESV message (e.g., an RTS/CTS pair) from a link in $\mathcal{C}(i)$ before the (T_i+1)th control mini-slot, it will not be included in $\mathbf{x}(t)$ and will not transmit an RESV message. Link i will set $x_i(t) = 0$.
3. IF link i does not hear an RESV message from any link in $\mathcal{C}(i)$ before the (T_i+1)th control mini-slot, it will send an RESV message to all links in $\mathcal{C}(i)$ at the beginning of the (T_i+1)th control mini-slot.
   — If there is a collision, link i will set $x_i(t) = 0$.
   — If there is no collision, link i will set $x_i(t) = 1$.
4. If $x_i(t) = 1$, link i will transmit a packet in the data slot.

In the above algorithm, $\lfloor x \rfloor^+ = x$ if $x \geq 0$ and $\lfloor x \rfloor^+ = 0$ if $x < 0$, $\text{Uniform}[0, W-1]$ is an integer random variable uniformly selected from $[0, W-1]$. Each control slot can be thought of as B frames, with each frame consisting of W mini-slots. A link is assigned a frame based on the log of its queue length, and the W mini-slots within a frame are used to resolve contentions. Hence, a control slot of D-GMS consists of $W \times B$ mini-slots, and links with empty queues will not compete for the channel in this timeslot.

Now we are ready to present a *hybrid* Q-CSMA algorithm that is both throughput-optimal and has very good delay performance. The basic idea behind the algorithm is as follows. For links with weight greater than a threshold w_0, the Q-CSMA procedure is applied first to determine their states; for other links, the D-GMS procedure is applied next to determine their states. To achieve this, a control slot is divided into W_0 mini-slots that are used to perform Q-CSMA for links whose weight is greater than w_0 and $W_1 \times B$ mini-slots that are used to implement D-GMS among the remaining links. Each link i uses a one-bit memory NA_i to record whether any of its conflicting links becomes active due to the Q-CSMA procedure in a timeslot. This information is used in constructing the schedule in the next timeslot.

Hybrid Q-CSMA (at Link i in Timeslot t)

IF $w_i(t) > w_0$ **(Q-CSMA Procedure)**

1.1 Link i selects a random backoff time

$$T_i = \text{Uniform}[0, W_0 - 1].$$

1.2 If link i hears an INTENT message from a link in $\mathcal{C}(i)$ before the (T_i+1)th control mini-slot, then it will set $x_i(t) = x_i(t-1)$ and go to Step 1.4.

1.3 If link i does not hear an INTENT message from any link in $\mathcal{C}(i)$ before the (T_i+1)th control mini-slot, it will send an INTENT message to all links in $\mathcal{C}(i)$ at the beginning of the (T_i+1)th control mini-slot.
- If there is a collision, link i will set $x_i(t) = x_i(t-1)$.
- If there is no collision, link i will decide its state as follows:
  if no links in $\mathcal{C}(i)$ were active due to the Q-CSMA procedure in the previous data slot, i.e., $NA_i = 0$
  $x_i(t) = 1$ with probability p_i, $\quad 0 < p_i < 1$;
  $x_i(t) = 0$ with probability $\bar{p}_i = 1 - p_i$.
  else
  $x_i(t) = 0$.

1.4 If $x_i(t) = 1$, link i will send an RESV message to all links in $\mathcal{C}(i)$ at the beginning of the (W_0+1)th control mini-slot. It will set $NA_i = 0$ and transmit a packet in the data slot.
If $x_i(t) = 0$ and link i hears an RESV message from any link in $\mathcal{C}(i)$ in the (W_0+1)th control mini-slot, it will set $NA_i = 1$; otherwise, it will set $NA_i = 0$.

IF $w_i(t) \leq w_0$ **(D-GMS Procedure)**

2.1 If link i hears an RESV message from any link in $\mathcal{C}(i)$ in the (W_0+1)th control mini-slot, it will set $NA_i = 1$ and $x_i(t) = 0$ and keep silent in this timeslot.

[1]We thank L. Jiang for this observation.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 16,2025 at 12:21:03 UTC from IEEE Xplore. Restrictions apply.

Otherwise, link i will set $NA_i = 0$ and select a random backoff time

$$T_i = (W_0 + 1) + W_1 \times \lfloor B - \log_b (q_i(t) + 1) \rfloor^+ + \text{Uniform}[0, W_1 - 1]$$

and wait for T_i control mini-slots.

2.2 If link i hears an RESV message from a link in $\mathcal{C}(i)$ before the $(T_i + 1)$th control mini-slot, it will set $x_i(t) = 0$ and keep silent in this timeslot.

2.3 If link i does not hear an RESV message from any link in $\mathcal{C}(i)$ before the $(T_i + 1)$th control mini-slot, it will send an RESV message to all links in $\mathcal{C}(i)$ at the beginning of the $(T_i + 1)$th control mini-slot.
— If there is a collision, link i will set $x_i(t) = 0$.
— If there is no collision, link i will set $x_i(t) = 1$.

2.4 If $x_i(t) = 1$, link i will transmit a packet in the data slot.

Remark 4: The $(W_0 + 1)$th control mini-slot (called *transition mini-slot*, which occurs between the first W_0 mini-slots and the last $W_1 \times B$ mini-slots) is reserved for all the links that have not been scheduled so far to conduct carrier sensing. In this mini-slot, those links that have already been scheduled (due to the Q-CSMA procedure) will send an RESV message so their neighbors can sense and record this information in their NA bits.

Remark 5: Suppose that the link weights are chosen as in Section III-A, i.e., $w_i(t) = f_i(q_i(t))$ is an increasing function of $q_i(t)$. Thus, $w_i(t) \gtrless w_0$ is equivalent to $q_i(t) \gtrless q_0$, where $q_0 = f^{-1}(w_0)$ is the *queue-length threshold*.

Remark 6: The control overhead of the hybrid Q-CSMA algorithm is $W_0 + 1 + W_1 \times B$ per timeslot. As in the (pure) D-GMS algorithm, links with empty queues will keep silent throughout the timeslot.

Let $L = \{i \in E : w_i(t) > w_0\}$ be the set of links for which the Q-CSMA procedure is applied (in timeslot t), and $L^c = E \setminus L$. Let $\mathbf{x}_L(t) = (x_i(t) : i \in L)$ be the transmission schedule of the links in L. Note that in the hybrid Q-CSMA algorithm, scheduling links in L^c will not affect the Q-CSMA procedure because those links will be scheduled after the links in L, and their transmissions will not be recorded by their neighboring links in the NA bits. Therefore, under fixed link weights and activation probabilities (so L is also fixed), $\mathbf{x}_L(t)$ evolves as a DTMC. Furthermore, using similar arguments as in the proofs of Propositions 1 and 3, we have

Proposition 4: If $W_0 \geq 2$, then the DTMC describing the evolution of the transmission schedule $\mathbf{x}_L(t)$ is reversible and has the following product-form stationary distribution:

$$\pi(\mathbf{x}_L) = \frac{1}{Z_L} \prod_{i \in \mathbf{x}_L} \frac{p_i}{\bar{p}_i} \tag{14}$$

$$Z_L = \sum_{\mathbf{x}_L \in \mathcal{M}_L} \prod_{i \in \mathbf{x}_L} \frac{p_i}{\bar{p}_i} \tag{15}$$

where $\mathcal{M}_L$ is the set of feasible schedules restricted to L.

Assuming a timescale separation property that the DTMC of $\mathbf{x}_L(t)$ is in steady state in every timeslot, we can establish the throughput optimality of the hybrid Q-CSMA algorithm in the following proposition.

Proposition 5: For each link $i \in L$, we choose its activation probability $p_i = e^{w_i(t)}/(e^{w_i(t)} + 1)$, where the link weights $w_i(t)$'s are appropriate functions of the queue lengths as in Section III-A. Then, the hybrid Q-CSMA algorithm is throughput-optimal.

Proof: Write $\mathbf{x}(t) = (\mathbf{x}_L(t), \mathbf{x}_{L^c}(t))$, where $\mathbf{x}_A(t) = (x_i(t) : i \in A)$ for any set $A \subseteq E$. Recall that MWS selects a maximum-weight schedule $\mathbf{x}^*(t)$ such that

$$\begin{aligned} w(\mathbf{x}^*(t)) &= w(\mathbf{x}^*_L(t)) + w(\mathbf{x}^*_{L^c}(t)) \\ &= \max_{\mathbf{x} \in \mathcal{M}} \sum_{i \in \mathbf{x}} w_i(t) =: w^* \end{aligned}$$

where $w(\mathbf{x}_A(t)) =: \sum_{i \in \mathbf{x}_A(t)} w_i(t)$. It is clear that

$$w(\mathbf{x}^*_L(t)) \leq \max_{\mathbf{x}_L \in \mathcal{M}_L} w(\mathbf{x}_L) =: w^*_L. \tag{16}$$

For any ϵ such that $0 < \epsilon < 1$, when $\|\mathbf{w}(t)\|_\infty \geq (2|E|w_0/\epsilon)$ (so L is not empty), we have

$$w^*_L \geq \max_i w_i(t) \geq \frac{2|E|w_0}{\epsilon}.$$

Therefore

$$w(\mathbf{x}^*_{L^c}(t)) \leq |E|w_0 \leq \frac{\epsilon}{2} w^*_L. \tag{17}$$

Using similar arguments as in the proof of Proposition 2, we can show that for any ϵ and δ such that $0 < \epsilon, \delta < 1$, if the queue lengths are large enough, then with probability greater than $1 - \delta$, the Q-CSMA procedure chooses $\mathbf{x}_L(t)$ such that

$$w(\mathbf{x}_L(t)) \geq \left(1 - \frac{\epsilon}{2}\right) \max_{\mathbf{x}_L \in \mathcal{M}_L} w(\mathbf{x}_L) = \left(1 - \frac{\epsilon}{2}\right) w^*_L.$$

In addition, if the queue lengths are large enough, then (17) holds. Therefore, since $w(\mathbf{x}_{L^c}(t)) \geq 0$, we have

$$\begin{aligned} w(\mathbf{x}(t)) &= w(\mathbf{x}_L(t)) + w(\mathbf{x}_{L^c}(t)) \\ &\geq (1 - \epsilon) w^*_L + \frac{\epsilon}{2} w^*_L \\ &\geq (1 - \epsilon) w(\mathbf{x}^*_L(t)) + w(\mathbf{x}^*_{L^c}(t)) \\ &\geq (1 - \epsilon) w^*. \end{aligned}$$

Hence, the hybrid Q-CSMA algorithm satisfies the condition of Theorem 1 and is throughput-optimal. ■

Remark 7: In the above algorithm, one can replace D-GMS by any other heuristic and still maintain throughput optimality. We simply use D-GMS because it is an approximation to GMS that is known to perform well in a variety of previous simulation studies. It is also important to recall our earlier observation that GMS is not a distributed algorithm, and hence, we have to resort to a distributed approximation.

VI. Simulation Results

In this section, we evaluate the performance of different scheduling algorithms via simulations, which include MWS (only for small networks), GMS (centralized), D-GMS, Q-CSMA, and the hybrid Q-CSMA algorithm. In addition, we have implemented a distributed algorithm to approximate maximal scheduling (called D-MS), which can be viewed as a synchronized slotted version of the IEEE 802.11 DCF with

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 16,2025 at 12:21:03 UTC from IEEE Xplore. Restrictions apply.

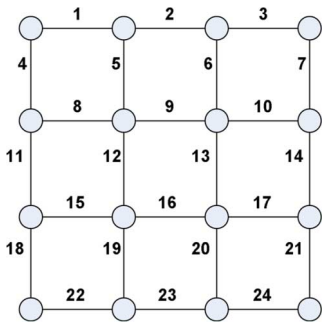


Fig. 1. 24-link grid network topology.

the RTS/CTS mechanism. D-MS is a special case of D-GMS presented in Section V with $B = 1$ so the backoff time of a link does not depend on its queue length.

D-MS (at Link i in Timeslot t)

1. Link i selects a random backoff time
$$T_i = \text{Uniform}[0, W-1]$$
and waits for T_i control mini-slots.
2. IF link i hears an RESV message from a link in $\mathcal{C}(i)$ before the (T_i+1)th control mini-slot, it will not be included in the transmission schedule $\mathbf{x}(t)$ and will not transmit an RESV message. Link i will set $x_i(t) = 0$.
3. IF link i does not hear an RESV message from any link in $\mathcal{C}(i)$ before the (T_i+1)th control mini-slot, it will send an RESV message to all links in $\mathcal{C}(i)$ at the beginning of the (T_i+1)th control mini-slot.
   — If there is a collision, link i will set $x_i(t) = 0$.
   — If there is no collision, link i will set $x_i(t) = 1$.
4. If $x_i(t) = 1$, link i will transmit a packet in the data slot. (Links with empty queues will keep silent in this timeslot.)

A. 24-Link Grid Network

We first evaluate the performance of the scheduling algorithms in a grid network with 16 nodes and 24 links as shown in Fig. 1. Each node is represented by a circle, and each link is illustrated by a solid line with a label indicating its index. Each link maintains its own queue. We assume 1-hop interference.

Consider the following four sets of links:

$$\mathcal{L}_1 = \{1, 3, 8, 10, 15, 17, 22, 24\}$$
$$\mathcal{L}_2 = \{4, 5, 6, 7, 18, 19, 20, 21\}$$
$$\mathcal{L}_3 = \{1, 3, 9, 11, 14, 16, 22, 24\}$$
$$\mathcal{L}_4 = \{2, 4, 7, 12, 13, 18, 21, 23\}.$$

Each set represents a (maximum-size) maximal schedule of the network: $\mathbf{M}_i = \mathbf{e}_{\mathcal{L}_i}$, where $\mathbf{e}_{\mathcal{L}_i}$ is a vector in which the components with indices in $\mathcal{L}_i$ are 1's and others are 0's. Then, we let the arrival rate vector be a convex combination of those maximal schedules scaled by ρ: $\boldsymbol{\lambda} = \rho \cdot \sum_{i=1}^{4} c_i \mathbf{M}_i$, $\mathbf{c} = [0.2, 0.3, 0.2, 0.3]$. Note that a convex combination of several maximum-size maximal schedules must lie on the boundary of the capacity region. Hence, the parameter ρ in [0, 1] can be viewed as the *traffic intensity*, with $\rho \to 1$ representing arrival rates approaching the boundary of the capacity region.

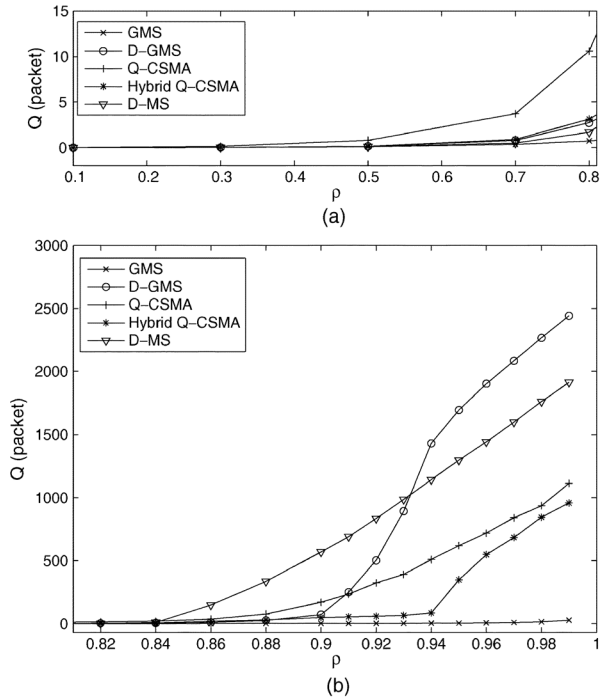


Fig. 2. Long-term average queue length per link in the 24-link grid network. (a) Low and moderate traffic intensity. (b) High traffic intensity.

The packet arrivals to each link i follow a Bernoulli process with rate λ_i independent of the packet arrival processes at other links. Each simulation experiment starts with all empty queues. For each algorithm under a fixed ρ, we take the average over 10 independent experiments, with each run being 10^5 timeslots. Due to the high complexity of MWS in such a large network, we do not evaluate it here.

For fair comparison, we choose a control overhead of 48 mini-slots for every distributed scheduling algorithm (which lies in the range of the backoff window size specified in IEEE 802.11 DCF [3]). The parameter setting of the scheduling algorithms is summarized as follows:

- D-MS: $W = 48$;
- D-GMS: $B = 3$, $W = 16$; $b = 8$;
- Q-CSMA: $W = 48$; link weight $w_i(t) = \log(0.1 q_i(t))$ and link activation probability $p_i = e^{w_i(t)}/(e^{w_i(t)} + 1)$;
- Hybrid Q-CSMA: $W_0 = 5$ for the Q-CSMA procedure, $B = 3$ and $W_1 = 14$ for the D-GMS procedure, plus one transition mini-slot. The queue-length threshold $q_0 = 100$. Other parameters are the same as Q-CSMA.

Remark 8: In Q-CSMA, we choose the link weight function $w_i(t) = \log(\alpha q_i(t))$ with a small constant α. The rationality is to make the link weights change much slower than the dynamics of the CSMA Markov chain (to satisfy the timescale separation assumption). We have tried several other choices for the link weight functions suggested in prior literature (such as $\alpha q_i(t)$ in [13] and $\log\log(q_i(t) + e)$ in [25]), but $\log(\alpha q_i(t))$ seems to give the best delay performance.

The performance of the scheduling algorithms is shown in Fig. 2, from which we can see the following (we have tested the algorithms under other traffic patterns, e.g., Poisson arrivals, different arrival rate vectors, and the results are similar).

- Under low to moderate traffic intensity, D-GMS and D-MS have very good delay performance (small long-term av-

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 16,2025 at 12:21:03 UTC from IEEE Xplore. Restrictions apply.

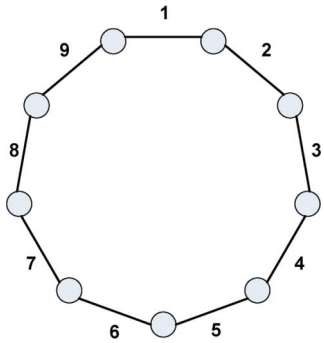


Fig. 3. Nine-link ring network topology.

erage queue length) and perform better than Q-CSMA. However, when the traffic intensity is high, the average queue length under D-GMS or D-MS blows up, and their delay performance becomes much worse than Q-CSMA.

- Hybrid Q-CSMA has the best delay performance among the distributed scheduling algorithms. It retains the stability property of Q-CSMA even under high traffic intensity while significantly reducing the delay of pure Q-CSMA. Note that when $\rho \to 1$, the performance of Hybrid Q-CSMA becomes close to pure Q-CSMA since the effect of the D-GMS procedure diminishes when the queue lengths of most links exceed the queue length threshold.
- Centralized GMS has excellent delay performance, but it is not throughput-optimal in general, as illustrated next.

B. Nine-Link Ring Network

Consider a nine-link ring network with 2-hop interference, as shown in Fig. 3. It was shown in [17] that GMS can only achieve 2/3 of the capacity region in this network. To see this, we construct a traffic pattern using the idea in [15]. Define $\mathcal{L}_i = \{i, (i+4) \bmod 9\}$, $1 \le i \le 9$. Starting with empty queues, in timeslot $9k + i$ ($k \in \mathbb{Z}, 1 \le i \le 9$), one packet arrives at each of the two links in $\mathcal{L}_i$, and, with probability ϵ, an additional packet arrives at each of the nine links. The average arrival rate vector is then $\boldsymbol{\lambda} = ((2/9) + \epsilon)\mathbf{e}$, where $\mathbf{e}$ is a vector with all components equal to 1. It has been shown in [15] that GMS will lead to infinite queue lengths under such a traffic pattern for all $\epsilon > 0$. On the other hand, we could use a scheduling policy as follows. Define

$$\tilde{\mathcal{L}}_1 = \{1, 4, 7\} \quad \tilde{\mathcal{L}}_2 = \{2, 5, 8\} \quad \tilde{\mathcal{L}}_3 = \{3, 6, 9\}$$

and let $\tilde{\mathbf{M}}_i = \mathbf{e}_{\tilde{\mathcal{L}}_i}$ for $1 \le i \le 3$. In timeslot $3k + i$ ($k \in \mathbb{Z}, 1 \le i \le 3$), the maximal schedule $\tilde{\mathbf{M}}_i$ is used. Hence, the average service rate vector is $\boldsymbol{\mu} = (1/3)\mathbf{e}$. When $0 < \epsilon < (1/9)$, $\boldsymbol{\lambda} < \boldsymbol{\mu}$, i.e., $\boldsymbol{\lambda}$ lies in the interior of the capacity region, but GMS cannot keep the network stable as we have seen above.

We evaluate the performance of the scheduling algorithms under the above traffic pattern. Each simulation experiment starts with all empty queues. For each algorithm under a fixed ϵ, we take the average over 10 independent experiments, with each run being 10^5 timeslots. We use exactly the same parameter setting as in Section VI-A. In Fig. 4 we can see that Q-CSMA and Hybrid Q-CSMA have a much lower delay than GMS, D-GMS (when $\epsilon \ge 0.03$) and D-MS (when $\epsilon \ge 0.05$).

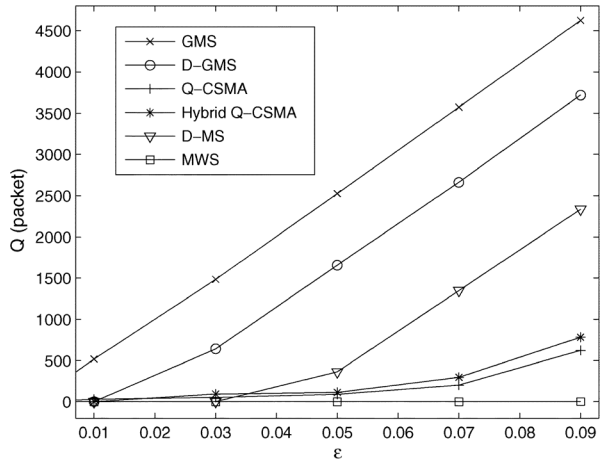


Fig. 4. Long-term average queue length per link in the nine-link ring network.

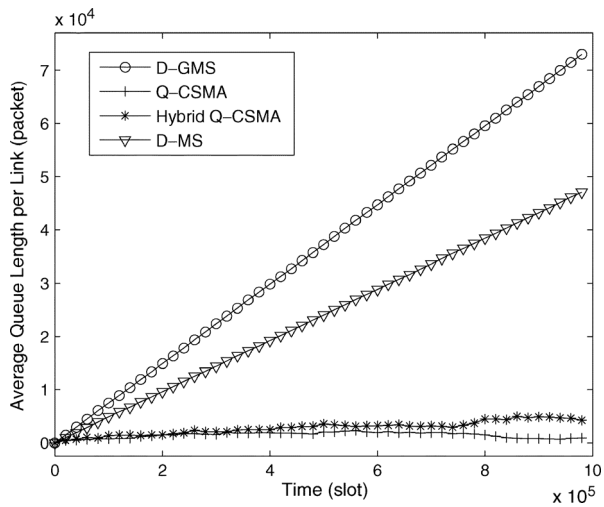


Fig. 5. Sample paths of average queue length per link: $\epsilon = 0.09$.

Fig. 5 shows that the average queue length increases linearly with the running time (number of timeslots) under D-GMS and D-MS, which implies they are not stable, while the average queue length is stable under Q-CSMA and Hyrbid Q-CSMA.

VII. Conclusion

In this paper, we proposed a discrete-time queue-length-based CSMA/CA protocol that leads to collision-free data transmission schedules. The protocol is provably throughput-optimal. The discrete-time formulation allows us to incorporate mechanisms to dramatically reduce the delay without affecting the theoretical throughput-optimality property. In particular, combining CSMA with distributed GMS leads to very good delay performance. We believe that it should be straightforward to extend our algorithms to be applicable to networks with multihop traffic and congestion-controlled sources (see [10], [19], and [26] for related surveys).

Appendix

A. Node-Based Implementation of Q-CSMA

When describing the Q-CSMA algorithm in Section IV, we treat every link as an entity, while in reality each link consists

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 16,2025 at 12:21:03 UTC from IEEE Xplore. Restrictions apply.

of a sender node and a receiver node. Both carrier sensing and transmission of data and control packets are actually conducted by those nodes. In this section, we provide details to implement Q-CSMA based on the nodes in the network.

For any node n in the network, we use $\mathcal{N}(n)$ to denote the neighborhood of n, which is the set of nodes that can hear the transmission of n. We assume *symmetry* in hearing: If $n' \in \mathcal{N}(n)$, then $n \in \mathcal{N}(n')$.

Let s_i and r_i be the sender node and receiver node of link i, respectively. If link i is included in the transmission schedule, then in the data slot, s_i will send a *data* packet to r_i, and r_i will reply an *ACK* packet to s_i. We assume that the data transmission from s_i to r_i is successful if no nodes in $\mathcal{N}(r_i)$ are transmitting in the same time; similarly, the ACK transmission from r_i to s_i is successful if no nodes in $\mathcal{N}(s_i)$ are transmitting in the same time. We also consider the node-exclusive constraint that two active links cannot share a common node. Therefore, in a *synchronized* data/ACK transmission system, the conflict set of link i is

$$\mathcal{C}(i) = \{j : j \text{ shares a common node with } i, \text{ or } s_j \in \mathcal{N}(r_i), \text{ or } r_j \in \mathcal{N}(s_i).\}.$$

In summary, two links i and j conflict with each other if they share a common node, or if simultaneous data transmissions at s_i and s_j will collide at r_i and r_j, or if simultaneous ACK transmissions at r_i and r_j will collide at s_i and s_j. Note that interference is assumed to be symmetric in our framework.

We say that a node is *active* in a timeslot if it is the sender node or receiver node of an active link. In a timeslot, each *inactive* node will conduct carrier sensing. It will determine whether there are some active sender nodes and some active receiver nodes in its neighborhood by sensing whether the channel is busy during the data transmission period and during the ACK transmission period, respectively. In this way, link i knows that no links in its conflict set are active in a timeslot if s_i and r_i do not belong to an active link other than i, s_i does not sense an active receiver node in $\mathcal{N}(s_i)$, and r_i does not sense an active sender node in $\mathcal{N}(r_i)$.

Similar to the RTS/CTS mechanism in the 802.11 MAC protocol, an INTENT message "sent" by a link consists of a *Request-To-Decide* (RTD) and a *Clear-To-Decide* (CTD) pair exchanged by the sender node and receiver node of the link. To achieve this, we further divide a control mini-slot into two sub-mini-slots. In the first sub-mini-slot, s_i sends an RTD to r_i. If r_i receives the RTD without a collision (i.e., no nodes in $\mathcal{N}(r_i)$ are transmitting in the same sub-mini-slot), then r_i will reply a CTD to s_i in the second sub-mini-slot. If s_i receives the CTD from r_i without a collision, then link $i = (s_i, r_i)$ will be added to the decision schedule. We choose the length of a sub-mini-slot such that an RTD or CTD sent by any node can reach its neighbors within one sub-mini-slot. Note that the exchange of an RTD/CTD pair between the sender and receiver of a link will silence all its conflicting links so those links will not be added to the decision schedule anymore.

Now we are ready to present the node-based Q-CSMA algorithm. Some additional one-bit memories maintained at node n (in timeslot t) are summarized as follows (each explanation corresponds to bit 1).

- $\mathrm{AS}_n(t)/\mathrm{AR}_n(t)$: n is *available* as the *sender*/*receiver* node for a link in the decision schedule $\mathbf{m}(t)$.
- $\mathrm{ACT}_n(t)$: n is *active* (as either a sender or receiver node).
- $\mathrm{NS}_n(t)/\mathrm{NR}_n(t)$: The *neighborhood* of n (i.e, $\mathcal{N}(n)$) has an active *sender*/*receiver* node.

Q-CSMA Algorithm (at Node n in Timeslot t)

1. At the beginning of the timeslot, node n sets $\mathrm{AS}_n(t) = 1$ and $\mathrm{AR}_n(t) = 1$.
   Let $L(n)$ be the set of links for which n is the sender node (i.e., $n = s_l, \forall l \in L(n)$). Node n randomly chooses one link in $L(n)$ (suppose link $i = (n, m)$ is chosen) and selects a backoff time T_i uniformly in $[0, W-1]$. Other links in $L(n)$ will not be included in $\mathbf{m}(t)$, so $x_l(t) = x_l(t-1), \forall l \in L(n) \setminus i$.
2. Throughout the control slot, if node n senses an RTD transmission not intended for itself (or a collision of RTDs) by a node in $\mathcal{N}(n)$, n will no longer be available as the receiver node for a link in $\mathbf{m}(t)$. Thus, node n will set $\mathrm{AR}_n(t) = 0$.
3. Before the (T_i+1)th control mini-slot, if node n senses a CTD transmission by a node in $\mathcal{N}(n)$ (or a collision of CTDs), n will no longer be available as the sender node for a link in $\mathbf{m}(t)$, and it will set $\mathrm{AS}_n(t) = 0$. In this case, link i will not be included in $\mathbf{m}(t)$ and $x_i(t) = x_i(t-1)$.
4. At the beginning of the (T_i+1)th control mini-slot, if $\mathrm{AS}_n(t) = 1$, node n will send an RTD to node m in the first sub-mini-slot. Node n will then set $\mathrm{AS}_n(t) = \mathrm{AR}_n(t) = 0$.
   4.1 If node m receives the RTD from node n without a collision and $\mathrm{AR}_m(t) = 1$, m will send a CTD to n in the second sub-mini-slot of the (T_i+1)th control mini-slot. Node m will then set $\mathrm{AS}_m(t) = \mathrm{AR}_m(t) = 0$. The CTD message also includes the carrier sensing information of node m in the previous timeslot (the values of $\mathrm{ACT}_m(t-1)$ and $\mathrm{NS}_m(t-1)$). Otherwise, no message will be sent.
   4.2 If node n receives the CTD message from node m without a collision, link $i = (n, m)$ will be included in $\mathbf{m}(t)$. Node n will decide link i's state as follows:
   if no links in $\mathcal{C}(i)$ were active in the previous data slot, i.e., $x_i(t-1) = 1$ or $\mathrm{ACT}_n(t-1) = \mathrm{ACT}_m(t-1) = \mathrm{NR}_n(t-1) = \mathrm{NS}_m(t-1) = 0$
   $x_i(t) = 1$ with probability p_i, $0 < p_i < 1$;
   $x_i(t) = 0$ with probability $\bar{p}_i = 1 - p_i$.
   else
   $x_i(t) = 0$.
   Otherwise, link i will not be included in $\mathbf{m}(t)$ and $x_i(t) = x_i(t-1)$.
5. In the data slot, node n takes one of the three different roles.
   — *Sender*: $x_l(t) = 1$ for some link $l = (n, m) \in E$. Node n will send a data packet to node m and set $\mathrm{ACT}_n(t) = 1$.
   — *Receiver*: $x_l(t) = 1$ for some link $l = (m, n) \in E$. Node n will send an ACK packet to node m (after it receives the data packet from m) and set $\mathrm{ACT}_n(t) = 1$.
   — *Inactive*: Node n sets $\mathrm{ACT}_n(t) = 0$ and conducts carrier sensing. Recall that data/ACK transmissions

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 16,2025 at 12:21:03 UTC from IEEE Xplore. Restrictions apply.

in our system are synchronized. Thus, node n will set $\mathrm{NS}_n(t) = 0$ if it senses no signal during the data transmission period, and set $\mathrm{NS}_n(t) = 1$ otherwise. Similarly, node n will set $\mathrm{NR}_n(t) = 0$ if it senses no signal during the ACK transmission period, and set $\mathrm{NR}_n(t) = 1$ otherwise.

Remark 9: Note that RTD and CTD are sent in two different sub-mini-slots. This provides an easy way to distinguish RTD and CTD (or collisions of RTDs and CTDs, respectively) without having to encode the packet type in a preamble bit of such a control packet (actually when a collision happens, a node cannot even check this "packet type" bit to distinguish RTD and CTD).

Proposition 6: $\mathbf{m}(t)$ produced by the node-based Q-CSMA algorithm is a feasible schedule. Let $\mathcal{M}_0$ be the set of decision schedules produced by the algorithm. If the window size $W \geq 2$, then $\cup_{\mathbf{m}\in\mathcal{M}_0}\mathbf{m} = E$, and the algorithm achieves the product-form distribution in Proposition 1.

Proof: The proof is similar to the proof of Lemma 3. Under the node-based Q-CSMA algorithm, link i will be included in the decision schedule $\mathbf{m}(t)$ if and only if its sender and receiver nodes successfully exchange an RTD/CTD pair in the control slot. This will silence all the receivers in $\mathcal{N}(s_i)$ and all the senders in in $\mathcal{N}(r_i)$ as well as nodes s_i and r_i, so no links in $\mathcal{C}(i)$ will be included in $\mathbf{m}(t)$. Hence, $\mathbf{m}(t)$ is a feasible schedule. Similarly, for any maximal schedule $\mathbf{m}$, we can check that $\mathbf{m}$ will be selected in the control slot with positive probability if the window size $W \geq 2$. Since the set of all maximal schedules will include all links, $\cup_{\mathbf{m}\in\mathcal{M}_0}\mathbf{m} = E$. Then, by Proposition 1 the node-based Q-CSMA algorithm achieves the product-form distribution. ∎

Therefore, the node-based Q-CSMA algorithm achieves the product-form distribution in Proposition 1 and is throughput-optimal if we let $p_i = e^{w_i(t)}/(e^{w_i(t)}+1), \forall i \in E$, where $w_i(t)$'s are appropriately chosen functions of the queue lengths.

B. Elimination of Hidden and Exposed Terminals

In IEEE 802.11 DCF, the RTS/CTS mechanism is used to reduce the *Hidden Terminal Problem*. However, even if RTS/CTS is used, the hidden terminal problem can still occur, as illustrated in Fig. 6, where we use S_i and R_i to denote the sender and receiver (in the sense of data transmission) of link i. Under 802.11, it is possible that S_2 is sending an RTS to R_2 while R_1 is returning a CTS to S_1 at the same time. R_1 cannot detect the RTS from S_2 since it is transmitting. Likewise, S_2 cannot detect the CTS from R_2. Therefore, both links 1 and 2 will be scheduled, which will cause a collision at R_1.

Our Q-CSMA algorithm can eliminate the hidden terminal problem due to the fact that the RTD/CTD messages are exchanged in a synchronized manner. Suppose T_i is the backoff expiration time of node S_i.

- $T_1 < T_2$: During the (T_1+1)th mini-slot, S_1 sends an RTD in the first sub-mini-slot, then in the second sub-mini-slot, R_1 returns a CTD, and link 1 will be included in the decision schedule. Since this CTD is not intended for S_2, S_2 disables its role as a sender of a link in the decision schedule, thus link 2 will not be included in the decision schedule.
- $T_1 = T_2$: During the $(T_1 + 1)$th mini-slot, both S_1 and S_2 send an RTD in the first sub-mini-slot. In this case, R_1 senses a collision of RTD and will not return a CTD. Thus, link 1 will not be included in the decision schedule, while link 2 will be included in the decision schedule.
- $T_1 > T_2$: Similarly, in this case link 2 but not link 1 will be included in the decision schedule.

Therefore, under synchronized RTD/CTD, the decision schedule is collision-free, which implies that the transmission schedule is collision-free if we start with a collision-free transmission schedule (e.g., the empty schedule), see Lemma 1. Hence, the hidden terminal problem is eliminated.

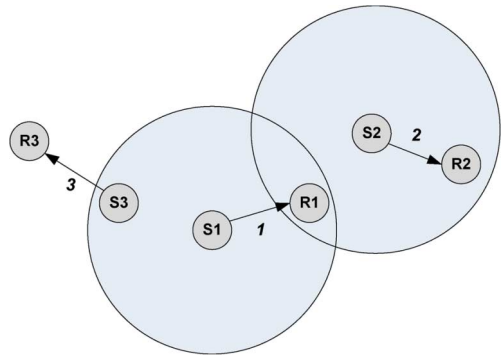


Fig. 6. Hidden and exposed terminal problems.

Another problem, known as the *Exposed Terminal Problem*, may also occur under 802.11. In Fig. 6, if S_1 sends an RTS to R_1, S_3 will receive this RTS and will be silenced under 802.11, which is unnecessary because the potential transmission of link 3 will not interfere with link 1. On the other hand, under Q-CSMA, if S_1 sends an RTD to R_1, S_3 will ignore this RTD and can still send an RTD to R_3. Therefore, both links 1 and 3 can be included in the decision schedule and the transmission schedule, thus avoiding the exposed terminal problem.

Note that the presence of hidden and exposed terminals not only leads to loss of efficiency, but also poses mathematical difficulties. In general, it is impossible to define a set of schedules that are consistent with both the definition of a feasible schedule defined in (1) and the capacity region defined in (2). Our RTD/CTD mechanism eliminates such problems.

REFERENCES

[1] U. Akyol, M. Andrews, P. Gupta, J. Hobby, I. Saniee, and A. Stolyar, "Joint scheduling and congestion control in mobile ad-hoc networks," in *Proc. IEEE INFOCOM*, April 2008, pp. 619–627.

[2] V. Bharghavan, A. Demers, S. Shenker, and L. Zhang, "MACAW: A media access protocol for wireless LAN's," in *Proc. ACM SIGCOMM*, 1994, pp. 212–225.

[3] G. Bianchi, "Performance analysis of the IEEE 802.11 distributed coordination function," *IEEE J. Sel. Areas Commun.*, vol. 18, no. 3, pp. 535–547, Mar. 2000.

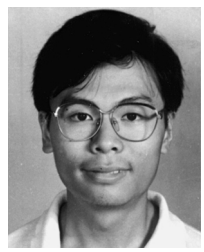
[4] R. R. Boorstyn, A. Kershenbaum, B. Maglaris, and V. Sahin, "Throughput analysis in multihop CSMA packet radio networks," *IEEE Trans. Commun.*, vol. COM-35, no. 3, pp. 267–274, Mar. 1987.

[5] C. Bordenave, D. McDonald, and A. Proutiere, "Performance of random medium access algorithms, an asymptotic approach," in *Proc. ACM SIGMETRICS*, Jun. 2008, pp. 1–12.

[6] P. Chaporkar, K. Kar, and S. Sarkar, "Throughput guarantees through maximal scheduling in wireless networks," in *Proc. 43rd Annu. Allerton Conf. Commun., Control, Comput.*, Sep. 2005, pp. 28–30.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 16,2025 at 12:21:03 UTC from IEEE Xplore. Restrictions apply.

[7] A. Dimakis and J. Walrand, "Sufficient conditions for stability of longest-queue-first scheduling: Second-order properties using fluid limits," *Adv. Appl. Probab.*, vol. 38, no. 2, pp. 505–521, 2006.
[8] M. Durvy and P. Thiran, "A packing approach to compare slotted and non-slotted medium access control," in *Proc. IEEE INFOCOM*, Apr. 2006, pp. 1–12.
[9] A. Eryilmaz, R. Srikant, and J. R. Perkins, "Stable scheduling policies for fading wireless channels," *IEEE/ACM Trans. Netw.*, vol. 13, no. 2, pp. 411–424, Apr. 2005.
[10] L. Georgiadis, M. Neely, and L. Tassiulas, "Resource allocation and cross-layer control in wireless networks," *Found. Trends Netw.*, vol. 1, no. 1, pp. 1–149, 2006.
[11] J. Ghaderi and R. Srikant, "On the design of efficient CSMA algorithms for wireless networks," Mar. 2010 [Online]. Available: http://arxiv.org/abs/1003.1364
[12] L. Jiang and J. Walrand, "Convergence analysis of a distributed CSMA algorithm for maximal throughput in a general class of networks," UC Berkeley, Berkeley, CA, Tech. Rep., Dec. 2008.
[13] L. Jiang and J. Walrand, "A distributed CSMA algorithm for throughput and utility maximization in wireless networks," in *Proc. 46th Annu. Allerton Conf. Commun., Control, Comput.*, Sep. 2008, pp. 1511–1519.
[14] L. Jiang and J. Walrand, "Approaching throughput-optimality in a distributed CSMA algorithm with contention resolution," UC Berkeley, Berkeley, CA, Tech. Rep., March 2009.
[15] C. Joo, X. Lin, and N. B. Shroff, "Understanding the capacity region of the greedy maximal scheduling algorithm in multi-hop wireless networks," *IEEE/ACM Trans. Netw.*, vol. 17, no. 4, pp. 1132–1145, Aug. 2009.
[16] F. Kelly, *Reversibility and Stochastic Networks*. Chichester, U.K.: Wiley, 1979.
[17] M. Leconte, J. Ni, and R. Srikant, "Improved bounds on the throughput efficiency of greedy maximal scheduling in wireless networks," *IEEE/ACM Trans. Netw.*, vol. 19, no. 3, pp. 709–720, Jun. 2011.
[18] S. C. Liew, C. Kai, J. Leung, and B. Wong, "Back-of-the-envelope computation of throughput distributions in CSMA wireless networks," *IEEE Trans. Mobile Comput.*, vol. 9, no. 9, pp. 1319–1331, Sep. 2010.
[19] X. Lin, N. B. Shroff, and R. Srikant, "A tutorial on cross-layer optimization in wireless networks," *IEEE J. Sel. Areas Commun.*, vol. 24, no. 8, pp. 1452–1463, Aug. 2006.
[20] J. Liu, Y. Yi, A. Proutiere, M. Chiang, and H. V. Poor, "Maximizing utility via random access without message passing," Microsoft Research, Tech. Rep., Sep. 2008.
[21] P. Marbach, A. Eryilmaz, and A. Ozdaglar, "Achievable rate region of CSMA schedulers in wireless networks with primary interference constraints," in *Proc. IEEE CDC*, Dec. 2007, pp. 1156–1161.
[22] J. Ni, B. R. Tan, and R. Srikant, "Q-CSMA: Queue-length based CSMA/CA algorithms for achieving maximum throughput and low delay in wireless networks," in *Proc. IEEE INFOCOM*, Mar. 2010, pp. 1–5.
[23] A. Proutiere, Y. Yi, and M. Chiang, "Throughput of random access without message passing," in *Proc. CISS*, Mar. 2008, pp. 509–514.
[24] S. Rajagopalan and D. Shah, "Distributed algorithm and reversible network," in *Proc. CISS*, Mar. 2008, pp. 498–502.
[25] S. Rajagopalan, D. Shah, and J. Shin, "Network adiabatic theorem: An efficient randomized protocol for contention resolution," in *Proc. ACM SIGMETRICS*, Jun. 2009, pp. 133–144.
[26] S. Shakkottai and R. Srikant, "Network optimization and control," *Found. Trends Netw.*, vol. 2, no. 3, pp. 271–379, 2007.
[27] L. Tassiulas and A. Ephremides, "Stability properties of constrained queueing systems and scheduling policies for maximal throughput in multihop radio networks," *IEEE Trans. Autom. Control*, vol. 37, no. 12, pp. 1936–1948, Dec. 1992.
[28] E. Vigoda, "A note on the Glauber dynamics for sampling independent sets," *J. Combin.*, vol. 8, pp. 1–8, 2001.
[29] X. Wang and K. Kar, "Throughput modelling and fairness issues in CSMA/CA based ad-hoc networks," in *Proc. IEEE INFOCOM*, Mar. 2005, vol. 1, pp. 23–34.
[30] A. Warrier, S. Janakiraman, and I. Rhee, "DiffQ: Practical differential backlog congestion control for wireless networks," in *Proc. IEEE INFOCOM*, Apr. 2009, pp. 262–270.
[31] X. Wu, R. Srikant, and J. R. Perkins, "Scheduling efficiency of distributed greedy scheduling algorithms in wireless networks," *IEEE Trans. Mobile Comput.*, vol. 6, no. 6, pp. 595–605, Jun. 2007.
[32] G. Zussman, A. Brzezinski, and E. Modiano, "Multihop local pooling for distributed throughput maximization in wireless networks," in *Proc. IEEE INFOCOM*, Apr. 2008, pp. 1139–1147.



Jian Ni (S'02–M'09) received the B.Eng. degree in automation from Tsinghua University, Beijing, China, in 2001, the M.Phil. degree in electrical and electronic engineering from the Hong Kong University of Science and Technology (HKUST), Hong Kong, in 2003, and the Ph.D. degree in electrical engineering from Yale University, New Haven, CT, in 2008.

He was a Postdoctoral Researcher with the Coordinated Science Laboratory, University of Illinois at Urbana–Champaign, from 2008 to 2010. He is currently a Research Staff Member with the IBM T. J. Watson Research Center, Yorktown Heights, NY. His research interests include performance analysis, algorithm design, communication networks, statistical inference, machine learning, and information extraction.



Bo (Rambo) Tan (S'11) received the B.E. degree in electronic engineering from Tsinghua University, Beijing, China, in 2007, and the M.S. degree in electrical and computer engineering from the University of Illinois at Urbana–Champaign (UIUC) in 2008, and is currently pursuing the Ph.D. degree in electrical and computer engineering at UIUC.

He is a Research Assistant with the Coordinated Science Laboratory, UIUC. He was a Research Intern with the Technicolor Paris Research Lab, Paris, France, from September to December 2009, and a Research Intern with Microsoft Research, Redmond, WA, from May to August 2010. His current research interests include online advertising, content distribution networks, wireless networks, P2P systems, and data centers, with emphasis on modeling, optimization, algorithm and protocol design, and performance analysis.



R. Srikant (S'90–M'91–SM'01–F'06) received the B.Tech. degree from the Indian Institute of Technology, Madras, India, in 1985, and the M.S. and Ph.D. degrees from the University of Illinois at Urbana–Champaign in 1988 and 1991, respectively, all in electrical engineering.

He was a Member of Technical Staff with AT&T Bell Laboratories, Holmdel, NJ, from 1991 to 1995. He is currently with the University of Illinois at Urbana–Champaign, where he is the Fredric G. and Elizabeth H. Nearing Professor with the Department of Electrical and Computer Engineering and a Research Professor with the Coordinated Science Laboratory. His research interests include communication networks, stochastic processes, queueing theory, information theory, and game theory.

Prof. Srikant was an Associate Editor of *Automatica*, the IEEE TRANSACTIONS ON AUTOMATIC CONTROL, and the IEEE/ACM TRANSACTIONS ON NETWORKING. He has also served on the Editorial Boards of special issues of the IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS and the IEEE TRANSACTIONS ON INFORMATION THEORY. He was the Chair of the 2002 IEEE Computer Communications Workshop in Santa Fe, NM, and a Program Co-Chair of IEEE INFOCOM 2007. He is a Distinguished Lecturer for the IEEE Communications Society for 2011–2012.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on January 16,2025 at 12:21:03 UTC from IEEE Xplore. Restrictions apply.